Operacijski sustavi (OS - 199570)



Nositelj: doc. dr. sc. Ivan Lorencin
Asistent: Luka Blašković, mag. inf.
Ustanova: Sveučilište Jurja Dobrile u Puli, Fakultet informatike u Puli

Fakultet informatike u Puli

(2) Zastavice CLI naredbi i Git CLI



Zastavice (*eng. flags ili options*) predstavljaju posebnu vrstu argumenata koji se koriste za izmjenu ili proširenje zadanog ponašanja naredbi u CLI sučelju.

Za razliku od argumenata koje smo dosad koristili prvenstveno za prosljeđivanje konkretnih podataka, poput putanja do datoteka ili direktorija, zastavice služe za parametrizaciju načina izvršavanja naredbe, odnosno za uključivanje dodatnih funkcionalnosti ili promjenu načina prikaza i obrade rezultata.

U pravilu se zastavice navode uz prefiks jedne crtice (`-`) za kratke oblike ili dviju crtica (`--`) za duže, opisne oblike. Njihova je osnovna svrha kontrola ponašanja naredbe, a ne prijenos sadržajnih podataka. Primjerice, zastavicom se može zatražiti detaljniji prikaz informacija, rekurzivna obrada direktorija ili sortiranje izlaza prema određenom kriteriju.

U ovom poglavlju studenti će se upoznati s najčešće korištenim zastavicama osnovnih bash naredbi iz cjeline OS1. Time će steći dublje razumijevanje fleksibilnosti CLI okruženja te razviti sposobnost učinkovitijeg i preciznijeg upravljanja radom u terminalu.

🆙 **Posljednje ažurirano: 13.9.2026.**

Sadržaj

- Operacijski sustavi (OS - 199570)
- (2) Zastavice CLI naredbi i Git CLI
  - Sadržaj

- 1. Uvod
- 2. Zastavice naredbe `ls`
  - 🚩Zastavice: `-a` i `-A`
  - 🚩Zastavica: `-l`
  - 🚩Zastavica: `-R`
  - 2.1 Ostale zastavice naredbe `ls`
  - 2.2 Tablica čestih zastavica naredbi `ls` i `tree` te primjeri kombiniranja
- 3. Zastavice naredbi `cd`, `pwd`, `mkdir` i `rmdir`
- Zadatak 1: Vježba osnovnih zastavica
- 4. Zastavice naredbi `cp`, `mv` i `rm`
  - 🚩Zastavica: `-i`
  - 🚩Zastavica: `-r`
  - 🚩Zastavica: `-v`
  - 🚩Zastavica: `-f`
  - 🚩Zastavica: `-n`
  - 4.1 Tablica čestih zastavica naredbi `cp`, `mv` i `rm`
- Zadatak 2: Vježba zastavica naredbi `cp`, `mv` i `rm`
- 5. Praktični primjer: Git CLI
  - 5.1 Instalacija Git CLI
  - 5.2 Osnovne Git CLI naredbe
    - 1. Način: Lokalni rad s Git-om
    - 2. Način: Rad s udaljenim repozitorijem
  - 5.3 Zastavice Git CLI naredbi
- Zadaci za Vježbu 2

1. Uvod

Na prošlim vježbama naučili smo osnovne bash naredbe za rad s datotekama i direktorijima unutar CLI sučelja. CLI omogućuje korisnicima interakciju s operacijskim sustavom putem tekstualnih naredbi, a interpretaciju samih naredbi obavlja ljuska (*eng. shell*).

Vidjeli smo što su **apsolutna** i **relativna** putanja te kako ih koristimo kao argumente za naredbe. Također, naučili smo kako navigirati kroz direktorije, stvarati nove datoteke i direktorije te ih premještati, preimenovati, kopirati i brisati kroz odgovarajuće naredbe.

U nastavku su navedene osnovne naredbe koje smo obradili, zajedno s opisima:

- `pwd` - ispisuje apsolutnu putanju trenutnog (radnog) direktorija
- `ls` - ispisuje sadržaj direktorija
- `cd` - mijenja trenutni direktorij
- `mkdir` - stvara novi direktorij
- `touch` - stvara novu datoteku
- `rm` - briše datoteku
- `rmdir` - briše direktorij (ako je prazan)
- `cp` - kopira datoteku ili direktorij
- `mv` - premješta datoteku ili direktorij
- `echo` - ispisuje tekst u terminalu ili preusmjerava tekst u datoteku

U ovoj skripti studenti će se upoznati s konceptom **opcija**, odnosno **zastavica** (*eng. options/flags*).

U *bashu*, **zastavice** ili **opcije** posebni su argumenti koji modificiraju ponašanje naredbi.

Zastavice se obično koriste kako bi se naredbama dodale funkcionalnosti ili kako bi se **promijenili neki interni parametri naredbe**.

Do sada nismo koristili zastavice, već smo naredbe nadopunjavali argumentima (npr. putanjama do datoteka i direktorija).

Argumenti (parametri) naredbi obično se koriste za prosljeđivanje podataka naredbi i navodimo ih nakon same naredbe:

```
→ naredba argument1 argument2 argument3 ... argumentN
```

Primjeri argumenata:

```
→ ls /mnt/c/Users

→ cp datoteka1.txt /mnt/c/Users

→ rm /mnt/c/Users/datoteka1.txt
```

Zastavice (opcije) se koriste za izmjenu ponašanja naredbi i navodimo ih nakon inicijalne naredbe, a obično se označavaju s `-` ili `--` (crtica ili dvostruka crtica):

- `-z` (kratka zastavica/kratki format)
- `--zastavica` (duga zastavica/dugi format)

```
→ naredba -a --zastavica2 -n ... --zastavicaN # uočite da zastavice mogu biti kratke (-a, `-n`) ili duge (--zastavica2, --zastavicaN)
```

Osim toga, vrlo često kombiniramo argumente i zastavice:

```
→ naredba -z --zastavica2 argument1 argument2 # 2 zastavice i 2 argumenta
```

U pravilu se zastavice pišu **prije argumenata**, ali to nije uvijek slučaj. Neki CLI programi mogu imati različite konvencije i dozvoljavaju različite redoslijede zastavica i argumenata.

Dakle, moguće je i sljedeće:

```
→ naredba argument1 -z argument2 --zastavica2 argument3
# argument1 ovdje se odnosi na "naredba", dok se argumenti argument2 i argument3 odnose na zastavice -z odnosno --zastavica2
```

- izgleda zbunjujuće, ali bit će jasnije kod naredbe `tree` koju ćemo obraditi kasnije

Primjeri korištenja zastavica u kombinaciji s argumentima:

```
→ ls -l /mnt/c/Users

→ cp -r vazni_dokumenti /mnt/c/Users/Desktop

→ mv -i script.js /home/user/Desktop/zadaca
```

Sintaksu koju smo pokazali na prošlim vježbama sada ćemo proširiti dodavanjem zastavica. Naglasak će biti na zastavicama osnovnih naredbi za rad s datotekama i direktorijima.

> Krenimo sa zastavicama! 🚩🚩🚩

2. Zastavice naredbe `ls`

Sintaksa:

```
→ ls [FLAGS] <putanja>
```

- zastavice `[FLAGS]` koristimo **prije putanje** kako bismo modificirali ispis
- `<putanja>` može biti apsolutna ili relativna putanja do datoteke ili direktorija

🚩Zastavice: `-a` i `-A`

`-a` (zapamti kao "all") → ispisuje sve datoteke/direktorije, uključujući i one skrivene (koje počinju s `.`)

`-A` (zapamti kao "almost all") → ispisuje sve datoteke/direktorije, uključujući i one skrivene, ali **ne** uključuje posebne direktorije `.` i `..`

Primjer:

```
→ mkdir vjezba_2
→ touch vjezba_2/datoteka.txt
→ cd vjezba_2
→ ls
```



> Slika 1. Naredba `ls` bez zastavice

Kako bismo dodali skrivenu datoteku, jednostavno dodajemo datoteku s točkom na početku:

```
→ touch .skrivena_datoteka.txt

→ ls # ne vidimo skrivenu datoteku

→ ls -a # vidimo skrivenu datoteku ".skrivena_datoteka.txt"
```



> Slika 2. Naredba `ls` sa zastavicom `-a` ispisuje skrivene datoteke

Ako bolje pogledate, osim skrivene datoteke `.skrivena_datoteka.txt`, vidimo i dva posebna direktorija: `.` i `..`.

Ovo su **specijalni pokazivači na direktorije** koji predstavljaju:

- **trenutni** direktorij (`.`),
- **roditeljski** direktorij (`..`)

te se nalaze u svakom direktoriju datotečnog sustava. U kontekstu naredbe `cd`, `cd ..` koristimo kada se želimo vratiti u roditeljski direktorij. Za povratak u prethodni radni direktorij koristi se `cd -`.

Ove specijalne direktorije moguće je upotrebljavati i s drugim naredbama, kao što je `ls`.

Primjerice, ako se nalazimo unutar direktorija `vjezba_2` i želimo ispisati sadržaj roditeljskog direktorija, možemo koristiti:

```
→ ls ..
```

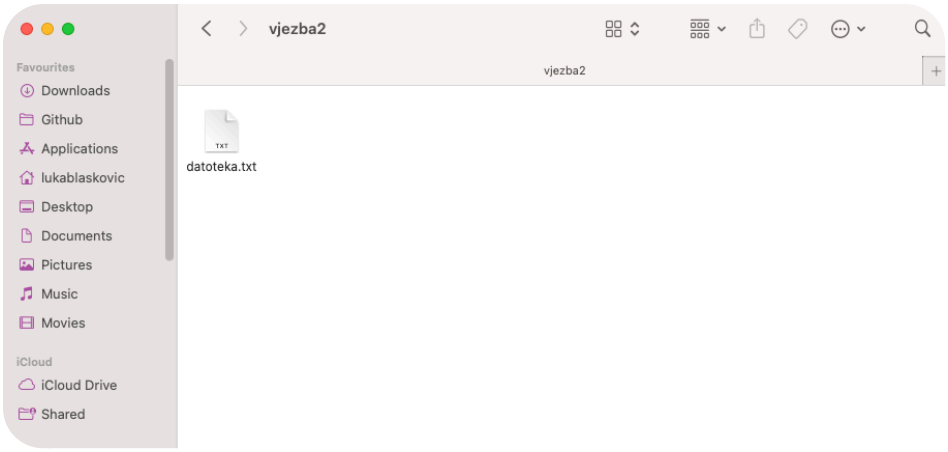
Ako bismo htjeli ispisati sadržaj roditeljskog direktorija roditeljskog direktorija, koristili bismo:

```
→ ls ../..
```

Također, sljedeće naredbe su ekvivalentne:

```
→ ls
→ ls .
```

Ako koristite zadane postavke unutar grafičkog sučelja (GUI), skrivene datoteke i direktoriji **obično nisu vidljivi**.



> Slika 3. Datoteka `.skrivena_datoteka.txt` često nije vidljiva u GUI-ju ako se koriste zadane postavke

- Kako prikazati skrivene datoteke na Windows OS GUI
- Kako prikazati skrivene datoteke/direktorije na macOS GUI
- Kako prikazati skrivene datoteke na Ubuntu/Linux OS GUI

Postoji varijanta ove zastavice s velikim slovom `-A` koja također ispisuje sve datoteke, ali ne ispisuje posebne direktorije `.` i `..`

```
→ ls -A # ne ispisuje posebne direktorije "." i ".."
```

Naredbe `open/xdg-open`

Kada koristimo OS s grafičkim sučeljem, možemo provjeriti rezultat izvršavanja naših `bash` naredbi i u GUI-ju.

Na **macOS-u** naredba `open` omogućuje otvaranje datoteke ili direktorija u GUI-ju.

Na većini **Linux** distribucija sličnu ulogu ima naredba `xdg-open`.

Primjerice, ako se nalazimo unutar direktorija `vjezba_2` i želimo otvoriti trenutni direktorij, možemo upotrijebiti pokazivač na trenutni direktorij `.`:

```
→ open . # macOS

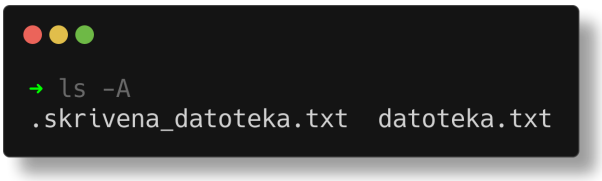
→ xdg-open . # Linux
```

U WSL okruženju naredba `open` u pravilu nije dostupna kao na macOS-u, pa se za otvaranje direktorija u Windows GUI-ju najčešće koristi `explorer.exe`.

```
→ explorer.exe .
```

Na ovaj način ćete pokrenuti program Windows File Explorer i otvoriti trenutni direktorij (`vjezba_2`) u GUI-ju

Prilikom korištenja `ls -A` nećete vidjeti `.` i `..`, ali ćete i dalje vidjeti skrivene datoteke koje počinju s `.`.

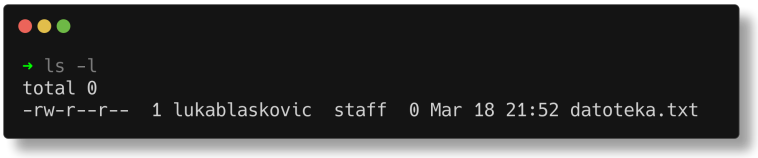


> Slika 4. Naredba `ls -A` ispisuje skrivene datoteke, ali ne ispisuje posebne direktorije `.` i `..`

🚩Zastavica: `-l`

`-l` (zapamti kao "long") → ispisuje detaljan prikaz datoteka i direktorija

```
→ cd vjezba_2
→ ls -l
```



> Slika 5. Naredba `ls -l` ispisuje detaljan prikaz datoteka i direktorija

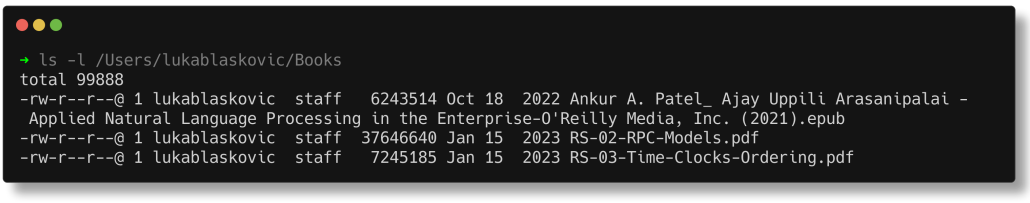
U detaljnom ispisu, svaki redak predstavlja jednu datoteku ili direktorij. Svaki redak sadrži sljedeće informacije:

1. **Tip datoteke i dozvole** - prvi znak označava tip datoteke (`d` → direktorij, `l` → simbolička veza, `-` → obična datoteka, ...), a sljedećih devet znakova predstavljaju dozvole za vlasnika, grupu i ostale korisnike (npr. `rwxr-xr--`)
2. **Broj čvrstih poveznica** (*eng. hard links*) - broj zapisa koji upućuju na isti inode, odnosno isti zapis unutar datotečnog sustava
3. **Vlasnik** - računalni korisnik koji je vlasnik datoteke/direktorija
4. **Grupa** - grupa kojoj datoteka/direktorij pripada
5. **Veličina** - veličina datoteke u bajtovima (`B`)
6. **Datum i vrijeme posljednje izmjene** - datum i vrijeme kada je datoteka/direktorij zadnji put izmijenjen
7. **Naziv direktorija/datoteke** - naziv direktorija ili datoteke

Primjer:

Želimo pogledati detaljan ispis direktorija `Books` koji se nalazi u *home* direktoriju korisnika `username`:

```
→ ls -l /mnt/c/Users/username/Books
```



> Slika 6. Naredba `ls -l` za detaljan ispis sadržaja direktorija `Books`

Vrijednost `total` koja se ispisuje na početku detaljnog ispisa ne predstavlja broj datoteka ili direktorija unutar radnog direktorija. Ona predstavlja **ukupan broj blokova koje zauzimaju prikazane stavke**, a točna veličina bloka ovisi o datotečnom sustavu i implementaciji naredbe `ls`.

Ako bismo željeli uključiti i skrivene datoteke u naš detaljni ispis, jednostavno dodajemo zastavicu `-a`:

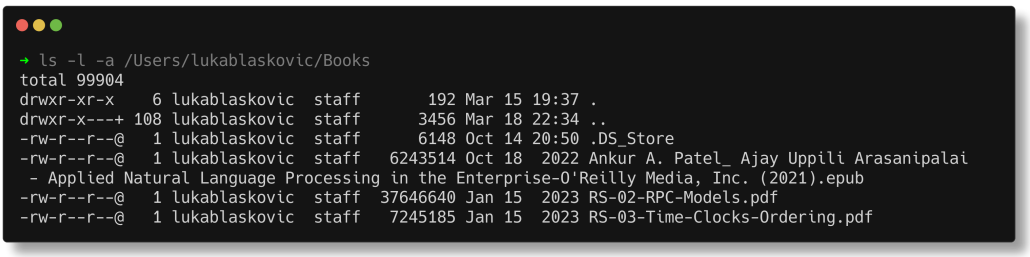
```
→ ls -l -a
```

🚨 **ZAPAMTI!** Redoslijed kratkih zastavica u pravilu nije bitan, sve dok pojedina zastavica ne očekuje dodatni argument.

```
→ ls -a -l
# ekvivalentno
→ ls -l -a
```

Ili spojeni zapis kratkih zastavica koji uključuje oba ova ponašanja (`-la`):

```
→ ls -la
# ili
→ ls -al
# Sve isto! (dok je riječ o kratkim zastavicama koje ne očekuju dodatne argumente)
```



> Slika 7. Naredba `ls -l -a` za detaljan ispis sadržaja direktorija `Books`, uključujući i skrivene datoteke

U ovoj skripti nećemo se detaljno baviti dozvolama, ali već možete uočiti neke osnovne informacije iz ovog ispisa:

- `-` (crtica) - obična datoteka
- `d` (direktorij) - direktorij
- `r` (read) - dozvola za čitanje
- `w` (write) - dozvola za pisanje
- `x` (execute) - dozvola za izvršavanje

> Oznake `+` i `@` koje se ponekad pojavljuju na kraju niza dozvola označavaju dodatne ACL (Access Control List) dozvole koje nisu prikazane u standardnom formatu.

Na primjeru od ranije:

- `drwxr-xr-x` - prvi znak `d` znači da se radi o **direktoriju**, a oznake `rwx` znače da vlasnik ima dozvolu za **čitanje, pisanje i izvršavanje**

- `-rw-r--r--` - uočite da je prvi znak `-`, što znači da se radi o **datoteci**, uočite i oznake `rw` što znači da vlasnik ima dozvolu za **čitanje i pisanje**, ali nema dozvolu za izvršavanje `x`

Za sada toliko! Detaljnije o dozvolama u cjelini OS5. 😎

🚩Zastavica: `-R`

`-R` (zapamti kao "recursive") → rekurzivno ispisuje sadržaj ugniježđenih direktorija (*eng. subdirectories*)

Primjer:

Unutar direktorija `vjezba_2` definirat ćemo sljedeću strukturu direktorija i datoteka:

```
→ mkdir dir_1_razina_1
→ mkdir dir_2_razina_1
→ mkdir dir_3_razina_1

→ touch datoteka_1_razina_1.txt
→ touch datoteka_2_razina_1.txt

→ touch dir_1_razina_1/datoteka_1_razina_2.js
→ touch dir_1_razina_1/datoteka_2_razina_2.html
→ mkdir dir_1_razina_1/dir_1_razina_2

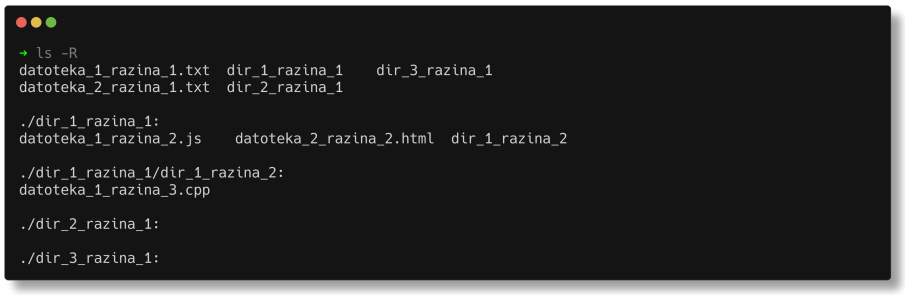
→ touch dir_1_razina_1/dir_1_razina_2/datoteka_1_razina_3.cpp
```

Očekujemo sljedeću strukturu:

```
[ 224]  .
├── [   0]  datoteka_1_razina_1.txt         # 1. razina
├── [   0]  datoteka_2_razina_1.txt         # 1. razina
├── [ 160]  dir_1_razina_1                  # 1. razina
│   ├── [   0]  datoteka_1_razina_2.js      # 2. razina
│   ├── [   0]  datoteka_2_razina_2.html    # 2. razina
│   └── [  96]  dir_1_razina_2              # 2. razina
│       └── [   0]  datoteka_1_razina_3.cpp # 3. razina
├── [  64]  dir_2_razina_1                  # 1. razina
└── [  64]  dir_3_razina_1                  # 1. razina
```

Naredbom `ls -R` ispisujemo sadržaj direktorija `vjezba_2` **rekurzivno**:

```
→ ls -R
```



> Slika 8. Naredba `ls -R` rekurzivno ispisuje sadržaj ugniježđenih direktorija i datoteka

Naredba `tree`

Digresija: Naredba `tree` također ispisuje sadržaj direktorija rekurzivno, ali u obliku stabla. Ova naredba ponekad nije dostupna na svim operacijskim sustavima, ali se može vrlo jednostavno instalirati.

Možete provjeriti dostupnost naredbe `tree` na vašem sustavu pokretanjem:

```
→ tree --version
```

Ako nije instalirana, na Ubuntu/Linux OS možete je instalirati naredbom:

```
→ sudo apt install tree
# ili
→ brew install tree # za macOS s Homebrew upraviteljem paketa
```

> 💡 **Hint**: Naredba sudo doslovno znači superuser do. Riječ je o jednoj od najčešće korištenih CLI naredbi u Unix-like sustavima (Linux, macOS). Omogućuje pokretanje naredbe ili skripte s ovlastima superkorisnika (root), odnosno s najvišim sigurnosnim privilegijama u sustavu.

Sintaksa:

```
→ tree [FLAGS] <putanja>
```

Primjer korištenja naredbe `tree`:

```
→ tree
```

Bez zastavica i argumenata, naredba `tree` **rekurzivno ispisuje sadržaj trenutnog direktorija u obliku stabla**.

```
.
└── datoteka.txt
```

Ako bismo htjeli vidjeti skrivenu datoteku `.skrivena_datoteka.txt`, dodajemo zastavicu `-a`:

```
→ tree -a
```

Rezultat:

```
.
├── .skrivena_datoteka.txt
└── datoteka.txt
```

Što uočavate? Zastavica `-a` kod naredbe `tree` također uključuje skrivene datoteke, ali ne i posebne direktorije `.` i `..`. Ipak, na početku ispisa vidimo `.` koji predstavlja trenutni direktorij iz kojeg pozivamo CLI naredbu.

Prema tome, možete uočiti da implementacije CLI programa, kao što je to `tree`, mogu uključivati srodne zastavice koje imaju slično ponašanje kao i kod drugih naredbi, ali nisu nužno identične.

Ako biste htjeli ispisati "stablo" *home* (matičnog) direktorija, mogli biste koristiti:

```
→ tree ~
```

> 💡 **Hint**: `~` (tilda) je ugrađeni prečac koji se u ljusci proširuje na korisnikov home direktorij. Ne znate gdje se nalazi na tipkovnici? Google it

> **Oprez!** Pokretanjem ove naredbe krenut ćete rekurzivno ispisivati sav sadržaj vašeg home direktorija, što može rezultirati vrlo velikim i skupim izlazom. **Kako biste prekinuli izvršavanje naredbe**, koristite kraticu `Ctrl + C` odnosno `Cmd + C` na macOS-u.

Kako bismo reducirali ispis na nekoliko razina, možemo koristiti zastavicu `-L` (zapamti kao "level"):

```
→ tree -L 2 ~ # ispisuje sadržaj home direktorija do 2 razine ugniježđenosti
```

Što je sad ovo! Ova naredba izgleda čudno jer smo proslijedili naizgled 3 argumenta: `-L`, `2` i `~`. Međutim, `-L` je zastavica koja očekuje dodatni argument koji specificira broj razina, a `~` je zaseban argument koji predstavlja putanju do home direktorija.

Moguće je napisati i obrnutim redoslijedom, bitno je da se pridržavamo sintakse gdje se zastavica `-L` koristi s odgovarajućim argumentom:

```
→ tree ~ -L 2 # isto kao iznad
```

Želimo uključiti i skrivene datoteke u naš ispis - nema problema!

```
→ tree -a -L 2 ~
```

Ako rastavimo naredbu iznad na dijelove, imamo:

- `tree` - naredba koja ispisuje sadržaj direktorija u obliku stabla
- `-a` - zastavica koja uključuje skrivene datoteke
- `-L 2` - zastavica s argumentom koja ograničava ispis na 2 razine ugniježđenosti
- `~` - argument koji predstavlja putanju do home direktorija

---

2.1 Ostale zastavice naredbe `ls`

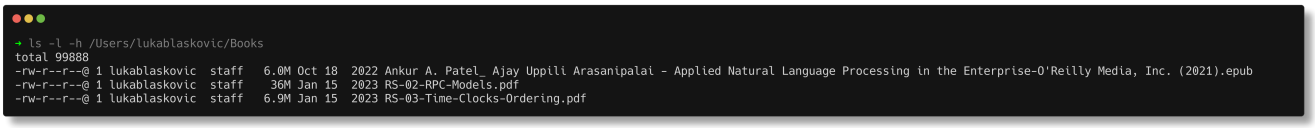
Naredba `ls` ima mnogo zastavica; pokazat ćemo još nekoliko korisnih koje se često koriste.

- `-h` (zapamti kao “human”) → ispisuje veličine datoteka u ljudski čitljivom formatu (npr. `1K`, `36M`, `3G`). Najčešće se kombinira sa zastavicom `-l`.

```
→ ls -l -h /mnt/c/Users/username/Books

# ili

→ ls -lh /mnt/c/Users/username/Books # kombiniranje zastavica -l i -h u -lh
```



> Slika 9. Naredba `ls -l -h` ispisuje veličine datoteka u “ljudski čitljivom” formatu

- `-t` (zapamti kao “time”) → sortira datoteke po vremenu zadnje izmjene (od **najnovijih** prema **najstarijima**).
- `-S` (zapamti kao “Size”) → sortira datoteke po memorijskoj veličini (od **najvećih** prema **najmanjima**).
- `-1` (zapamti kao “one”) → ispisuje svaku datoteku u zasebnom redu
- `--color` (u GNU `ls`) → obojit će ispis prema tipu stavke; točne boje ovise o temi i postavkama terminala, na macOS-u je tipično koristiti zastavicu `-G` umjesto `--color`.

> 💡**Hint**: Kod naredbe `ls` gotovo je sve zastavice moguće međusobno kombinirati (npr. `-la`, `-lR`, `-aR`).

Primjeri kombiniranja zastavica:

1. Detaljni ispis svih datoteka i direktorija u trenutnom direktoriju, sortiranih prema vremenu zadnje izmjene (od najnovijih prema najstarijima):

```
→ ls -lt # kombiniranjem: -l (detaljan ispis), -t (sortiranje prema vremenu)
```

2. Detaljni ispis svih datoteka i direktorija u trenutnom direktoriju, uključujući i skrivene datoteke, sortiranih po veličini (od najvećih prema najmanjima):

```
→ ls -laS # kombiniranjem: -l (detaljan ispis), -a (sve datoteke), -S (sortiranje po
veličini)
```

3. Rekurzivni detaljni ispis svih datoteka i direktorija u trenutnom direktoriju, sortiranih prema vremenu zadnje izmjene (od najnovijih prema najstarijima):

```
→ ls -lR -t # kombiniranjem: -l (detaljan ispis), -R (rekurzivno), -t (sortiranje prema
vremenu zadnje izmjene od najnovijih prema najstarijima)
```

4. Običan ispis svih datoteka i direktorija u trenutnom direktoriju, svaki zapis u novom redu:

```
→ ls -1
```

5. Ispis svih datoteka i direktorija, uključujući skrivene ali bez `.` i `..` direktorija, obojene, svaki zapis u novom redu:

```
# nalazimo se unutar direktorija: "vjezba_2"
→ ls -A --color -1
```



> Slika 10. Naredba `ls -A --color -1` ispisuje sve datoteke i direktorije, uključujući skrivene, ali bez `.` i `..`, obojeno i sa svakom stavkom u zasebnom retku

2.2 Tablica čestih zastavica naredbi `ls` i `tree` te primjeri kombiniranja

Zastavica	Primjeri kombiniranja	Objašnjenje
`-l`	`ls -l` / `ls -l -h` / `ls -lh`	Detaljan popis datoteka/direktorija s dozvolama, vlasnikom, grupom, veličinom i datumom izmjene i drugim detaljima.
`-a`	`ls -a`	Prikazuje sve stavke, uključujući skrivene datoteke i direktorije te posebne direktorije `.` i `...`
`-A`	`ls -A`	Prikazuje skrivene datoteke, ali bez `.` i `..`.
`-h`	`ls -h` / `ls -l -h` / `ls -lh`	Prikazuje veličine u čitljivom formatu (`B`, `K`, `M`, `G`).
`-t`	`ls -t` / `ls -l -t` / `ls -lt`	Sortira datoteke prema vremenu izmjene (najnovije prve).

Zastavica	Primjeri kombiniranja	Objašnjenje
`-r`	`ls -r` / `ls -l -r` / `ls -lr`	Prikazuje popis datoteka/direktorija obrnutim redoslijedom.
`-R`	`ls -R` / `ls -l -R` / `ls -lR`	Rekurzivno ispisuje sadržaj svih poddirektorija.
`-1`	`ls -1`	Prikazuje popis datoteka/direktorija u jednom stupcu (svaku stavku u posebnom retku).
`-X`	`ls -X` / `ls -l -X` / `ls -lX`	Sortira datoteke prema ekstenziji.
`--color`	`ls --color`	Prikazuje ispis u boji prema tipu stavke (GNU `ls`).
`-L`	`tree -L 2`	Ograničava ispis naredbe `tree` na određeni broj razina ugniježđenosti.
`-a`	`tree -a`	Prikazuje sve stavke, uključujući skrivene datoteke i direktorije.
`-h`	`tree -h`	Prikazuje veličine u čitljivom formatu (`B`, `K`, `M`, `G`).

> 💡 **Napomena:** zastavice treba izvježbati i koristiti u praksi kako bi se bolje razumjele i zapamtile. Također, nisu sve zastavice dostupne u svakoj implementaciji naredbe `ls`; primjerice `--color` tipičan je za GNU `ls`.

> 💡 **Hint**: Kod većine naredbi pomoć je moguće otvoriti naredbom `man`, npr. `man ls` za detaljne upute o naredbi `ls` i dostupnim zastavicama. Za Bash ugrađene naredbe, poput `cd` i `pwd`, često je praktično koristiti i `help cd` ili `help pwd`. Iz prikaza `man` izlazi se pritiskom tipke `q`.

3. Zastavice naredbi `cd`, `pwd`, `mkdir` i `rmdir`

Naredba `cd` ima opcije `-L` i `-P`, ali se u osnovnom radu koriste rijetko pa ih ovdje nećemo detaljno obrađivati.

Naredba `pwd` također ima dvije opcije (`-L` i `-P`), no također nam nisu jako zanimljive.

Naredbe `mkdir` i `rmdir` podržavaju korisnu zastavicu `-p`. Kod naredbe `mkdir` ona omogućuje stvaranje više ugniježđenih direktorija u jednom koraku, dok kod naredbe `rmdir` omogućuje brisanje više ugniježđenih direktorija odjednom.

Na prošlim vježbama, rekli smo da ne možemo napraviti direktorij unutar nepostojećeg direktorija, odnosno:

```
# ne možemo stvoriti direktorij "test" unutar nepostojećeg direktorija "files_manipulation"
→ mkdir files_manipulation/test
```

- Ipak, zastavica `-p` omogućit će nam upravo to.

```
→ mkdir -p files_manipulation/test

# pa i više ugniježđenih direktorija
# Primjer: stvaranje ugniježđenih direktorija "dir1/dir2/dir3"
→ mkdir -p dir1/dir2/dir3
```

Ekvivalentno možemo koristiti i naredbu `rmdir -p` za brisanje ugniježđenih direktorija, ako su prazni:

```
→ rmdir -p dir1/dir2/dir3 # pokušat će obrisati dir3, zatim dir2, a zatim dir1, ali samo ako su prazni
```

Zadatak 1: Vježba osnovnih zastavica

- ne predaje se (samo za vježbu)

1. Stvorite direktorij `vjezba_ls` i unutar njega stvorite sljedeću strukturu direktorija i datoteka koristeći isključivo `mkdir` i `touch` naredbe:
   - ugniježđene direktorije stvorite koristeći odgovarajuću zastavicu naredbe `mkdir`
   - datoteke možete stvoriti i jednom naredbom `touch`, jednostavno navodeći sve datoteke koje želite stvoriti kao argumente

```
[  96]  . vjezba_ls
└── [ 128]  faks
    ├── [ 256]  1_semestar
    │   ├── [   0]  diferencijalni_i_integralni_racun.txt
    │   ├── [   0]  logika_i_diskretna_mat.txt
    │   ├── [   0]  multimedijalni_sustavi.txt
    │   ├── [   0]  osnove_ikt.txt
    │   ├── [   0]  osnove_podatkovne_znanosti.txt
    │   └── [   0]  programiranje.txt
    └── [ 256]  2_semestar
        ├── [   0]  baze_podataka_1.txt
        ├── [   0]  geometrija_i_linearna_algebra.txt
        ├── [   0]  informacijski_sustavi.txt
        ├── [   0]  matematicka_analiza.txt
        ├── [   0]  operacijski_sustavi.txt
        └── [   0]  programiranje_u_skriptnim_jezicima.txt
```

2. Unutar direktorija `vjezba_ls` ispišite rekurzivno sadržaj direktorija `faks`, u boji, svaku stavku u zasebnom redu bez skrivenih datoteka.

Primjer:

```
faks

./faks
1_semestar
2_semestar

./faks/1_semestar:
diferencijalni_i_integralni_racun.txt
logika_i_diskretna_mat.txt
multimedijalni_sustavi.txt
osnove_ikt.txt
osnove_podatkovne_znanosti.txt
programiranje.txt

./faks/2_semestar:
baze_podataka_1.txt
geometrija_i_linearna_algebra.txt
informacijski_sustavi.txt
matematicka_analiza.txt
operacijski_sustavi.txt
programiranje_u_skriptnim_jezicima.txt
```

3. Unutar direktorija `vjezba_ls` ispišite **detaljan** prikaz datoteka iz 1. semestra, sortiran po datumu zadnje izmjene, od najnovije prema najstarijoj.

Primjer:

```
total 0
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:08 osnove_podatkovne_znanosti.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:07 osnove_ikt.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:07 multimedijalni_sustavi.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:07 programiranje.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:07 logika_i_diskretna_mat.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:07 diferencijalni_i_integralni_racun.txt
```

4. U drugi semestar dodajte tajni kolegij koji je skrivena datoteka i **detaljno** ispišite sve datoteke iz 2. semestra uključujući i skrivene datoteke, bez `.` i `..` direktorija.

```
total 0
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:15 .tajni_kolegij.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:09 programiranje_u_skriptnim_jezicima.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:08 baze_podataka_1.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:08 geometrija_i_linearna_algebra.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:08 informacijski_sustavi.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:08 matematicka_analiza.txt
-rw-r--r--  1 lukablaskovic  staff  0 Mar 19 00:08 operacijski_sustavi.txt
```

5. Ako se nalazite u direktoriju `faks/2_semestar`, objasnite dva načina kako biste dodali novu datoteku `.tajni_kolegij.txt` u direktorij `faks/1_semestar` bez prebacivanja u taj direktorij, tj. bez korištenja naredbe `cd`.
6. Jednom *bash* naredbom ispišite stablasti prikaz vašeg *home* direktorija do 3 razine ugniježđenosti, uključujući skrivene datoteke, ali bez `.` i `..` direktorija. Nakon toga, pokušajte CLI naredbom otvoriti taj direktorij u GUI-ju.

4. Zastavice naredbi `cp`, `mv` i `rm`

U ovom dijelu pokrit ćemo zastavice naredbi `cp` (kopiranje), `mv` (premještanje) i `rm` (brisanje), koje omogućuju precizniju kontrolu nad njihovim ponašanjem.

Ove tri naredbe imaju dosta zastavica koje se ponavljaju i međusobno su slične, stoga ćemo ih obraditi zajedno.

Sintaksa:

```
→ cp [FLAGS] <izvor> <odredište>

→ mv [FLAGS] <izvor> <odredište>

→ rm [FLAGS] <putanja>
```

- zastavice `[FLAGS]` u pravilu navodimo **prije** izvora i odredišta
- `<izvor>`, `<odredište>` i `<putanja>` mogu biti apsolutne ili relativne putanje; kod `cp` i `mv` to mogu biti i datoteke i direktoriji

🚩Zastavica: `-i`

`-i` (zapamti kao "interactive") → prije kopiranja/premještanja/brisanja datoteke, **zastavica će pitati korisnika za potvrdu**. Zastavica je korisna u slučajevima kada želimo izbjeći slučajnu izmjenu važnih datoteka, pogotovo kad radimo s više datoteka odjednom ili kad radimo s datotekama koje su važne za rad operacijskog sustava.

- zastavica `-i` unutar `cp` će pitati korisnika za potvrdu **samo ako datoteka već postoji na odredištu** (*eng. overwrite*)
- zastavica `-i` unutar `mv` će pitati korisnika za potvrdu **samo ako datoteka već postoji na odredištu**
- zastavica `-i` kod naredbe `rm` traži potvrdu prije svakog brisanja stavke, a u kombinaciji s `-r` može tražiti potvrdu i pri ulasku u direktorije te pri njihovu uklanjanju.

Primjeri:

```
→ cp -i datoteka.txt /mnt/c/Users/username/Desktop # naredba će pitati korisnika prije kopiranja datoteke samo ako "datoteka.txt" već postoji na odredištu

# ili

→ mv -i datoteka.txt /mnt/c/Users/username/Desktop # pitaj korisnika prije premještanja datoteke na odredište ako "datoteka.txt" već postoji

# ili

→ rm -i datoteka.txt # pitaj korisnika prije brisanja datoteke svaki put
```

Korisnik odgovara na pitanje s `y` (*yes*) ili `n` (*no*), odnosno **unosom odgovarajućeg slova** i pritiskom tipke `Enter`.

Primjer:

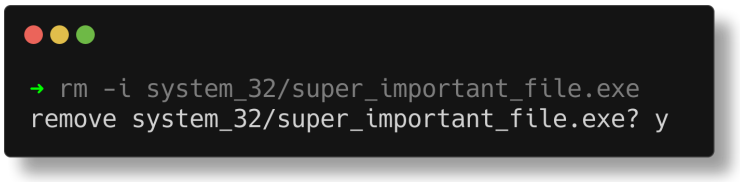
```
→ mkdir system_32 # bez brige, nije stvarni direktorij!
→ touch system_32/super_important_file.exe

→ rm -i system_32/super_important_file.exe
```

Nakon izvršavanja naredbe, korisnik će dobiti sljedeći ispis:

```
rm: remove system_32/super_important_file.exe?
```

- Ako korisnik odgovori s `y` i pritisne `Enter`, datoteka će biti obrisana
- Ako korisnik odgovori s `n` **ili bilo kojim drugim znakom** i pritisne `Enter`, datoteka neće biti obrisana



> Slika 11. Naredba `rm -i` traži potvrdu prije brisanja datoteke

🚩Zastavica: `-r`

`-r` (zapamti kao "recursive") → kopira ili briše direktorij i **sav njegov sadržaj rekurzivno**. Pomoću ove zastavice moguće je kopirati ili obrisati direktorije i sve datoteke/poddirektorije unutar njih u jednom koraku.

- ovo ponašanje je već zadano kod naredbe `mv` (premještanje) pa iz tog razloga nema zastavice `-r` kod iste
- zastavicu `-r` je moguće pisati i velikim slovom: `-R` (pripazi: kod naredbe `ls` ove zastavice nisu ekvivalentne, a kod naredbi `cp` i `rm` najčešće jesu - uvijek je dobro provjeriti u dokumentaciji CLI programa koji koristite)

Kod **kopiranja** (`cp`) rekli smo da možemo kopirati određenu datoteku ili direktorij iz mjesta `<izvor>` u mjesto `<odredište>`:

```
→ cp <izvor> <odredište>
```

- gdje izvor i odredište mogu biti relativne ili apsolutne putanje

Do sada smo vidjeli primjere gdje kopiramo:

- jedan direktorij u drugi direktorij rekurzivno → (`cp -r dir dir2`)
- jednu datoteku u drugi direktorij → (`cp datoteka.txt dir`)
- jednu datoteku u drugu datoteku (s istim ili različitim nazivom) → (`cp datoteka.txt datoteka2.txt`)

Međutim, **što ako želimo kopirati cijeli direktorij, uključujući sav njegov sadržaj**, u drugi direktorij? Tada je potrebno koristiti rekurzivnu zastavicu `-r`:

```
→ cp -r <izvor> <odredište>
```

Primjer s rekurzivnim kopiranjem:

```
→ mkdir vjezba_cp_r

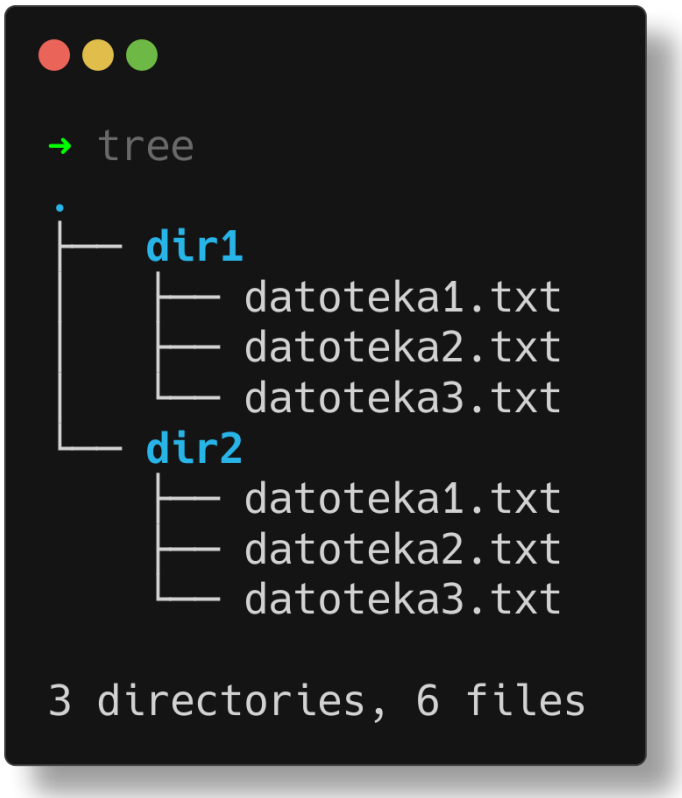
→ mkdir vjezba_cp_r/dir1

→ cd vjezba_cp_r/dir1
→ touch datoteka1.txt datoteka2.txt datoteka3.txt

→ cd ..
→ mkdir dir2

# Primjer: kopirat ćemo ukupan sadržaj direktorija "dir1" u direktorij "dir2" (ne premještamo sam direktorij)
→ cp -r dir1 dir2
```

Rezultat:



> Slika 13. Naredba `cp -r` kopira cijeli direktorij "dir1" i sav njegov sadržaj u direktorij "dir2" (ne dobivamo direktorij unutar direktorija)

Kod **premještanja** (`mv`), rekurzivno ponašanje je zadano i ne navodi se eksplicitno ovom zastavicom:

```
→ mv <izvor> <odredište> # nema zastavice -r
```

Primjer:

```
→ mkdir vjezba_mv

→ mkdir vjezba_mv/dir1

→ cd vjezba_mv/dir1
→ touch script1.js script2.js script3.js

→ cd ..
→ mkdir dir2

# nalazimo se unutar direktorija: "vjezba_mv"
# Primjer: premještanje cijelog direktorija sa sadržajem u drugi direktorij (dir1 -> dir2)
→ mv dir1 dir2 # premješta "dir1" u "dir2" (doslovno cijeli direktorij)
```

Rezultat:



> Slika 14. Naredba `mv` premješta cijeli direktorij "dir1" i sav njegov sadržaj u direktorij "dir2" (dobivamo direktorij unutar direktorija)

Wildcard `*`

Ipak, ako bismo htjeli premjestiti samo sadržaj direktorija `dir1` u `dir2`, a ne cijeli direktorij, moramo koristiti zamjenski znak (*wildcard*) `*`.

Općenito, zamjenski znakovi (*wildcards*) koriste se kada želimo radnju izvršiti nad više datoteka odjednom. Konkretno, `*` predstavlja **sve neskrivene stavke** unutar nekog direktorija.

VAŽNO! Wildcard `*` nije zastavica niti argument CLI naredbe, već poseban znak unutar putanje.

Sintaksa:

```
→ naredba dir1/dir2/* # wildcard nije zastavica, već poseban znak unutar putanje
```

```
→ mv dir1/* dir2 # premješta sve neskrivene stavke iz "dir1" u "dir2"

# "dir1" je sad prazan
# "dir2" sadrži sve neskrivene stavke iz "dir1"
```

- Ako postoji previše datoteka unutar direktorija, wildcard `*` može dati grešku: “Argument list too long”.

Kod **brisanja** (`rm`) moramo biti posebno oprezni jer rekurzivno brisanje direktorija i njegovog sadržaja može biti **nepovratno**.

```
rm -r <direktorij>
```

Primjer rekurzivnog brisanja:

```
→ mkdir vjezba_rm_r

→ mkdir vjezba_rm_r/dir1

→ cd vjezba_rm_r/dir1

→ touch cache1.txt cache2.txt cache3.txt cache4.txt

→ cd ..
→ ls -l dir1

# nalazimo se unutar direktorija: "vjezba_rm_r"
# Primjer: rekurzivno brisanje direktorija "dir1" i svih datoteka unutar njega
→ rm -r dir1
```

> 🚨**Oprez**: **Rekurzivno brisanje direktorija i njegovog sadržaja može biti opasno**, stoga je potrebno pažljivo provjeriti navodite li ispravan direktorij prije brisanja. Dobra je praksa kombinirati zastavicu `-r` sa zastavicom `-i`, koja u tom slučaju pita korisnika za potvrdu. Mnoge moderne implementacije `rm` imaju i zaštitu od brisanja korijenskog direktorija ili drugih kritičnih lokacija datotečnog sustava.

Primjer rekurzivnog brisanja s potvrdom:

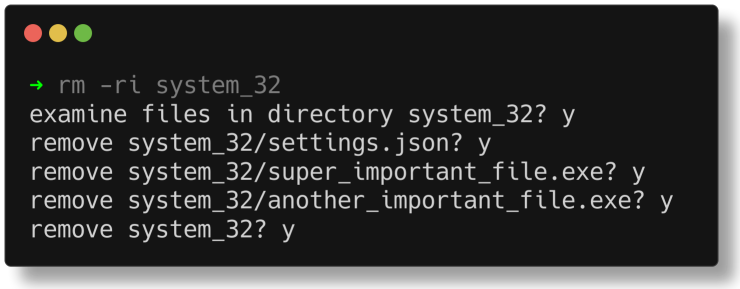
```
→ mkdir system_32

→ touch system_32/super_important_file.exe
→ touch system_32/another_important_file.exe
→ touch system_32/settings.json

# Primjer: rekurzivno brisanje direktorija "system_32" s potvrdom za svaku datoteku
→ rm -ri system_32
```

Na ovaj način naredba `rm` će:

- pitati korisnika za pregled svakog direktorija (`examine`)
- pitati korisnika je li siguran u brisanje svake datoteke (`remove`)
- pitati korisnika je li siguran u brisanje ukupnog direktorija (`remove directory`)
- korisnik odgovara s `y` ili `n` i pritiskom tipke `Enter` na isti način kao što smo pokazali ranije



> Slika 12. Naredba `rm -ri` traži potvrdu prije brisanja svake datoteke i direktorija

🚩Zastavica: `-v`

`-v` (zapamti kao "verbose") → **ispisuje detalje o rezultatu naredbe** koja se izvršava (npr. ispisuje datoteke koje se kopiraju, premještaju ili brišu)

Izraz *verbose* općenit je pojam koji označava detaljniji ispis informacija. U kontekstu naredbi `cp`, `mv` i `rm`, zastavica `-v` ispisuje **detalje o rezultatu radnje** koja se izvršava, npr. naziv datoteke koja se kopira, premješta ili briše.

```
→ cp -v datoteka.txt /mnt/c/Users/username/Desktop # ispisuje detalje o kopiranju datoteke

→ mv -v datoteka.txt /mnt/c/Users/username/Desktop # ispisuje detalje o premještanju datoteke

→ rm -v datoteka.txt # ispisuje detalje o brisanju datoteke
```

Primjer s detaljima o kopiranju:

```
→ mkdir vjezba_v

→ touch vjezba_v/datoteka1.txt

→ cp -v vjezba_v/datoteka1.txt vjezba_v/datoteka2.txt # kopira datoteku i preimenuje je, ispisuje detalje o radnji
```



> Slika 13. Zastavica `-v` ispisuje detalje o promjeni, npr. `vjezba_v/datoteka1.txt -> vjezba_v/datoteka2.txt`

Primjer s detaljima o rekurzivnom brisanju i potvrdama:

```
→ mkdir vjezba_v_rm

→ cd vjezba_v_rm

→ touch spam1.txt spam2.txt spam3.txt spam4.txt

→ cd ..

# Primjer: kombinirat ćemo zastavice -v, -i i -r za upit prije brisanja svake datoteke
# i ispisati obrisanu datoteku/direktorij nakon svake operacije
→ rm -vir vjezba_v_rm
```



> Slika 14. Kombinacija zastavica `-v`, `-i` i `-r` ispisuje detalje o brisanju i traži potvrdu za svaku radnju

🚩Zastavica: `-f`

`-f` (zapamti kao "force") → **forsira izvršavanje naredbe** bez traženja potvrde. Kod naredbe `rm` pritom se potiskuje i poruka o pogrešci ako datoteka ne postoji. Ova zastavica može biti korisna, ali i opasna ako nismo pažljivi.

Zastavica `-f` se koristi kod naredbi `cp`, `mv` i `rm`:

- kada želimo obrisati **datoteke**, a uz `-r` i direktorije, bez potvrde i bez poruke o pogrešci ako ne postoje (`rm -f`)
- kada želimo prepisati postojeću ciljnu datoteku prilikom kopiranja bez interaktivnog upita (`cp -f`)
- kada želimo premjestiti datoteke i po potrebi prepisati postojeće bez interaktivnog upita (`mv -f`)

Zastavica `-f` **može dovesti do nepovratnog gubitka podataka**. Preporučuje se koristiti je samo kada ste sigurni da želite prisilno izvršiti operaciju i prethodno ste provjerili putanju te datoteke koje se brišu, premještaju ili kopiraju.

> 💡**Napomena**: Ponašanje zastavice `-f` može se djelomično razlikovati ovisno o konkretnoj implementaciji alata (`GNU coreutils`, BSD alati i sl.) te o eventualnim aliasima u *shellu*. To ponašanje ne mora izravno ovisiti o verziji *bash shell-a*.

Primjer brisanja bez potvrde:

```
→ mkdir vjezba_f

→ cd vjezba_f

→ touch osjetljiva_datoteka.txt

# Brisanje bez potvrde
→ rm -f osjetljiva_datoteka.txt
```

Kako bismo demonstrirali rad ove zastavice, moramo imati datoteke različitog sadržaja.

Upis u datoteku možemo napraviti pomoću naredbe `echo` i operatora `>`:

Sintaksa:

```
→ echo "string_sadrzaj" > datoteka.txt
```

- za sada je to dovoljno, a detalje ćemo obrađivati na sljedećim vježbama

Primjer kopiranja bez potvrde:

```
→ mkdir vjezba_cp_f

→ touch vjezba_cp_f/backup.log

→ echo "Stari podaci" > vjezba_cp_f/backup.log # sintaksa za upis u datoteku (radit ćemo ovo kasnije)

# Stvaramo novu datoteku s novim podacima
→ echo "Novi podaci" > novi_backup.log # sintaksa za upis u datoteku (radit ćemo ovo kasnije)

# Kopiramo i prepisujemo datoteku bez upozorenja
→ cp -f novi_backup.log vjezba_cp_f/backup.log
```

Primjer premještanja bez potvrde:

```
→ mkdir vjezba_mv_f

→ touch vjezba_mv_f/old_config.cfg

→ echo "Stara konfiguracija" > vjezba_mv_f/old_config.cfg

# Kreiramo novu konfiguracijsku datoteku
→ echo "Nova konfiguracija" > new_config.cfg

# Premještamo i prepisujemo bez upozorenja
→ mv -f new_config.cfg vjezba_mv_f/old_config.cfg
```

🚨**Opasna kombinacija zastavica** `-f` i `-r` može dovesti do rekurzivnog brisanja sadržaja direktorija bez potvrde.

Primjer rekurzivnog brisanja bez potvrde:

```
→ mkdir -p vjezba_rm_rf/temp

→ touch vjezba_rm_rf/temp/file1.txt vjezba_rm_rf/temp/file2.txt

# Brisanje cijelog direktorija bez upita
→ rm -rf vjezba_rm_rf
```

> 💡**Hint**: Ako niste sigurni u radnju, preporuka je izbjegavati `-f` ili koristiti zastavicu `-i` za potvrdu.

Još jednom: zastavica `-f` nije zadana sama po sebi; razlike u ponašanju najčešće dolaze od aliasa, postavki sustava ili konkretne implementacije naredbe.

🚩Zastavica: `-n`

`-n` (zapamti kao “no overwrite”) → sprječava prepisivanje postojećih datoteka prilikom kopiranja (`cp`) ili premještanja (`mv`). Ova zastavica je korisna kada **ne želimo izgubiti postojeće podatke slučajnim prepisivanjem**.

Zastavica `-n` će **preskočiti kopiranje/premještanje datoteke ako već postoji na odredištu** i koristimo ju:

- **kada kopiramo ili premještamo datoteke**, ali ne želimo prebrisati postojeće datoteke
- kada želimo **zaštititi stare verzije datoteka**
- kada ne želimo ručno potvrđivati svaku zamjenu (`-i`), već jednostavno **automatski spriječiti prepisivanje**

Ova zastavica na neki je način suprotnost zastavici `-f`.

Praktično je kombinirati `-n` i `-v` zastavice za bolju vidljivost i kontrolu nad naredbama.

Primjer s kopiranjem:

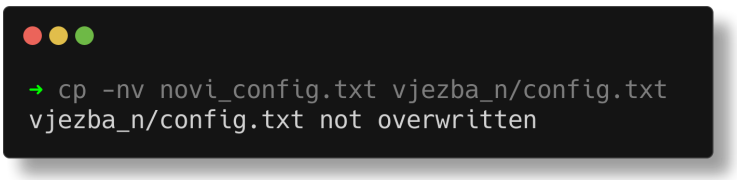
```
→ mkdir vjezba_n

→ echo "Prva verzija" > vjezba_n/config.txt

→ echo "Najnovija verzija" > novi_config.txt

# Kopiramo, ali ne prepisujemo ako "config.txt" već postoji
→ cp -n novi_config.txt vjezba_n/config.txt

# Kombiniranje s -v za ispis detalja
→ cp -nv novi_config.txt vjezba_n/config.txt
```



> Slika 15. Naredba `cp` s kombinacijom zastavica `-n` i `-v` ispisuje detalje o radnji i ne prepisuje datoteku ako već postoji

Primjer s premještanjem:

```
→ mkdir vjezba_mv_n

→ echo "Originalna verzija" > vjezba_mv_n/backup.txt

→ echo "Nova verzija" > novi_backup.txt

# Premještamo, ali ne prepisujemo ako "backup.txt" već postoji
→ mv -n novi_backup.txt vjezba_mv_n/backup.txt

# Kombiniranje s -v za ispis detalja
→ mv -nv novi_backup.txt vjezba_mv_n/backup.txt
```



> Slika 16. Naredba `mv` s kombinacijom zastavica `-n` i `-v` ispisuje detalje o radnji i ne prepisuje datoteku ako već postoji

4.1 Tablica čestih zastavica naredbi `cp`, `mv` i `rm`

Zastavica	Sintaksa	Objašnjenje
`-i`	`cp -i` / `mv -i` / `rm -i`	**Interaktivni način** – traži potvrdu prije prepisivanja (`cp`, `mv`) ili brisanja (`rm`) datoteke.
`-r`	`cp -r` / `rm -r`	**Rekurzivno kopira** (`cp`) ili briše (`rm`) direktorij i sve njegove poddirektorije i datoteke.
`-R`	`cp -R` / `rm -R`	U pravilu isto kao `-r` (za `cp` i `rm`). Kod `ls` ove zastavice nisu ekvivalentne.
`-v`	`cp -v` / `mv -v` / `rm -v`	**Prikazuje detalje** o izvršenim operacijama (`verbose`).
`-f`	`cp -f` / `mv -f` / `rm -f`	**Forsira izvršenje** – kod `rm` ne traži potvrdu i ne prijavljuje nepostojeću datoteku, a kod `cp` i `mv` prisilno prepisuje cilj kada je potrebno.

Zastavica	Sintaksa	Objašnjenje
`-n`	`cp -n` / `mv -n`	**Onemogućuje prepisivanje** postojećih datoteka
`-u`	`cp -u` / `mv -u`	Kopira ili premješta **samo ako je izvorna datoteka novija od ciljne ili ako ciljna ne postoji**.

> 💡 **Napomena:** Naredba `mv` radi s direktorijima bez posebne rekurzivne zastavice, pa nema opciju `-r`. Kod `rm` se `-r` koristi pažljivo jer može trajno obrisati velike količine podataka.

Zadatak 2: Vježba zastavica naredbi `cp`, `mv` i `rm`

- ne predaje se (samo za vježbu)

1. Stvorite direktorij `vjezba_cp_mv_rm` i unutar njega sljedeću strukturu direktorija koristeći isključivo naredbu `mkdir`.

- naredbu `mkdir` **smijete pozvati najviše 5 puta**. Hint: odgovarajuća zastavica

```
[ 160]  .
├── [ 128]  Documents
│   ├── [  64]  faks
│   └── [  64]  sve_ostalo
├── [ 160]  Games
│   ├── [  64]  action
│   ├── [  64]  puzzle
│   └── [  64]  sandbox
```

2. Koristeći naredbu `touch`, unutar direktorija `sve_ostalo` stvorite sljedeće datoteke:

```
salabahter_ikt.txt
salabahter_os.txt
salabahter_programiranje.txt
```

- stvorite novu datoteku `salabahter_ikt.txt` unutar direktorija `faks` i unesite u nju sadržaj `"Salabahter iz IKT-a"` (kroz CLI ili GUI)

- zatim odjednom kopirajte sav sadržaj direktorija `sve_ostalo` u direktorij `faks`, pri čemu se postojeća datoteka ne smije prepisati, te ispišite detalje o radnji

3. Unutar direktorija `Games` stvorite **svih 5 datoteka jednom naredbom**. Datoteke predstavljaju kratice (shortcut, s nastavkom `.lnk`) na igre koje pripadaju kategorijama `action`, `puzzle` i `sandbox`.

- `minecraft.lnk`
- `portal2.lnk`
- `DOOM.lnk`
- `the_witness.lnk`
- `the_legend_of_zelda.lnk`

Koristeći naredbe `cp` i `mv`, rasporedite datoteke u odgovarajuće direktorije i ispišite detalje o radnji. Neke igre pripadaju u više kategorija pa je u tim slučajevima potrebno kopirati datoteke, a ne samo premještati.

1. action i puzzle: `minecraft.lnk`
2. puzzle i action: `portal2.lnk`
3. action: `DOOM.lnk`
4. puzzle: `the_witness.lnk`
5. action, puzzle i sandbox: `the_legend_of_zelda.lnk`

Napišite odgovarajuću naredbu ili skup naredbi kojim ćete obrisati sve kratice unutar poddirektorija direktorija `Games`, uz potvrdu prije brisanja svake datoteke. Nakon brisanja direktoriji trebaju ostati netaknuti, ali prazni.

5. Praktični primjer: Git CLI

Git je distribuirani sustav za upravljanje verzijama (*eng. distributed version control system*) koji se koristi za praćenje promjena u datotekama tijekom razvoja softvera (ali i drugih projekata).

Razvio ga je Linus Torvalds 2005. godine za potrebe razvoja Linux kernela, a danas je jedan od najpopularnijih alata za upravljanje verzijama u svijetu softverskog razvoja.

Git omogućuje:

- praćenje povijesti promjena u projektu
- suradnju više developera na istom programskom kodu
- vraćanje projekta na ranije verzije (*rollback*)
- paralelni razvoj kroz grane (*branches*)

Za razliku od centraliziranih sustava (npr. SVN), **Git je distribuiran** - svaki korisnik ima cijeli repozitorij lokalno, uključujući kompletnu povijest.

Ključni pojmovi:

- **Repozitorij**: direktorij u kojoj Git prati promjene
- **Commit**: zapis (*eng. snapshot*) trenutnog stanja projekta
- **Branch**: paralelna linija razvoja unutar repozitorija
- **Merge**: spajanje promjena s jedne grane na drugu
- **Remote**: udaljeni repozitorij (npr. na GitHubu/GitLabu)



> Slika 17. Git CLI omogućuje upravljanje verzijama i suradnju na projektima

5.1 Instalacija Git CLI

Na osobnim računalima, Git je najsigurnije preuzeti ručno sa službene stranice i slijediti upute za instalaciju. Nakon instalacije, Git CLI je dostupan kroz terminal.

Git CLI je alat u naredbenom retku koji omogućuje interakciju s Git repozitorijima. Kroz Git CLI možemo izvršavati različite naredbe za upravljanje verzijama, grana, *commitova* i drugih aspekata Git repozitorija.

Git CLI nije dio *bash shell-a*, niti je sam *shell* program. Umjesto toga, radi se o zasebnom programu koji se može pozivati iz *bash shell-a* ili drugih terminala (kao što je to program `tree` koji smo koristili ranije). Git CLI se koristi za izvršavanje Git naredbi, a *bash shell* je okruženje u kojem se te naredbe izvršavaju.

Kako biste provjerili je li Git CLI ispravno instaliran, otvorite terminal i unesite:

```
→ git --version # duga zastavica :)
# ili
→ git -v # kratka zastavica :)
```

- ako vidite verziju, npr. `git version 2.53.0`, to znači da je Git CLI ispravno instaliran i spreman za korištenje

5.2 Osnovne Git CLI naredbe

Kada koristite Git, možete pristupiti poslu na dva načina:

1. Izraditi lokalni repozitorij na vašem računalu i koristiti Git CLI za upravljanje njime
2. Klonirati postojeći udaljeni repozitorij (npr. s GitHub-a) i koristiti Git CLI za upravljanje lokalnom kopijom

Kloniranje (eng. cloning) udaljenog repozitorija je proces stvaranja lokalne kopije repozitorija na vašem računalu. To vam omogućuje da radite s projektom lokalno, a zatim **sinkronizirate promjene** s udaljenim repozitorijem.

1. Način: Lokalni rad s Git-om

Mi ćemo prvo pokazati kako izraditi lokalni Git repozitorij i upravljati njime - bez povezivanja s udaljenim repozitorijem.

1. Korak

Stvorite novi prazan direktorij

```
→ cd ~ # ili bilo gdje drugdje

→ mkdir lokalni_git_projekt
```

2. Korak

Prebacite se u direktorij i inicijalizirajte Git repozitorij naredbom `git init`

```
→ cd lokalni_git_projekt

→ git init
```

Primjer rezultata: `Initialized empty Git repository in /Users/lukablaskovic/lokalni_git_projekt/.git/`

Uspješno ste izradili lokalni Git repozitorij! Provjerimo sadržaj direktorija:

```
→ ls
```

Nema ništa! To je zato što Git **pohranjuje sve informacije o verzijama i povijesti u skrivenom direktoriju** `.git`.

Mi smo naučili koristiti CLI zastavice, pa ćemo koristiti `ls -a` da vidimo i skrivene datoteke:

```
→ ls -a

# Rezultat:
.  ..  .git
```

> `.git` je skriveni direktorij koji Git koristi za pohranu svih informacija o verzijama, granama, *commitovima* i drugim aspektima repozitorija. Sadržaj `.git` direktorija je ključan za funkcioniranje Git repozitorija, ali ga **obično ne trebamo ručno mijenjati**.

Jednom kada imate `.git` direktorij, tada možete unutar roditeljskog direktorija (`lokalni_git_projekt`) stvarati datoteke, mijenjati ih i koristiti Git CLI naredbe za praćenje promjena, stvaranje *commitova* i druge radnje.

3. Korak

Stvorite datoteku `README.md` i unesite u nju sadržaj `#Moj lokalni Git projekt` (kroz CLI ili GUI)

VAŽNO: Ovo radite unutar direktorija `lokalni_git_projekt`, nikako unutar `.git` direktorija!

```
→ echo "#Moj lokalni Git projekt" > README.md # Stvori datoteku i upiši odmah sadržaj u nju

# ili

→ touch README.md # pa ručno unesite sadržaj kroz GUI i editor po želji
→ open README.md # odnosno xdg-open README.md ili explorer.exe .
```

Provjerite sadržaj direktorija

```
→ ls -a
```

Sadržaj datoteke možete brzo provjeriti naredbom `cat` ili kroz GUI:

```
→ cat README.md

# Rezultat: #Moj lokalni Git projekt
```

4. Korak

Sada kada imamo datoteku, možemo koristiti Git CLI naredbe za praćenje promjena i stvaranje *commitova*. Na primjer:

```
→ git add README.md # dodaj datoteku u staging area

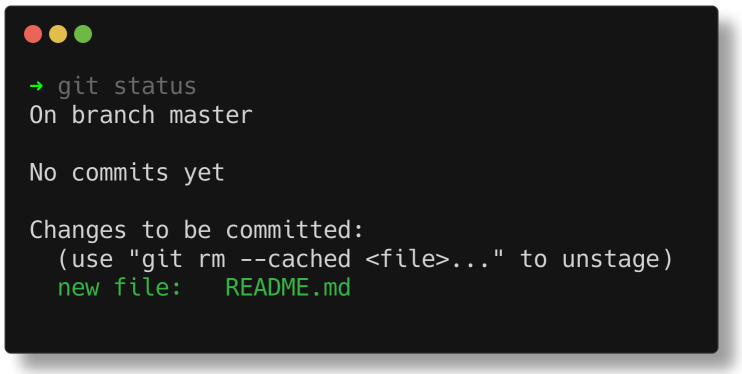
→ git add . # dodaj sve datoteke u staging area (ako ih je previše)
```

Staging area predstavlja privremenu zonu gdje se pripremaju promjene prije nego što se trajno zabilježe u povijesti projekta kroz *commit*.

> U većim projektima (koji imaju mnogo datoteka) često se koristi jednostavno `git add .` što znači: evidentiraj u *staging area* sve promjene u trenutnom direktoriju i njegovim poddirektorijima.

Kako biste provjerili koje su datoteke dodane u staging area, možete koristiti naredbu `git status`:

```
→ git status
```



> Slika 18. Naredba `git status` prikazuje koje su datoteke dodane u staging area i koje promjene nisu praćene. Za sada je `README.md` jedina datoteka u staging area, a drugih promjena nema.

5. Korak

Sljedeći korak je stvaranje *commita*, što znači da trajno bilježimo promjene koje su trenutno u staging area. To radimo naredbom `git commit`:

Git Commit predstavlja zapis trenutnog stanja projekta (*eng. snapshot*) i **uključuje obaveznu poruku** koja opisuje promjene koje su napravljene. Poruka se navodi pomoću zastavice `-m` koja očekuje argument u obliku stringa.

Sintaksa:

```
→ git commit -m "Poruka *commita*"
```

Obzirom da smo mi naučili osnovnu strukturu parsiranja *bash* naredbe, sada možete uočiti da `git` predstavlja naziv programa, `commit` je podnaredba (subcommand) koja specificira radnju, a `-m "Poruka *commita*"` je zastavica i njezin argument koji daje dodatne informacije o tome što se *commita*. Struktura je sljedeća:

```
→ naredba podnaredba zastavica argument_zastavice
```

Primjer stvaranja *commita*:

```
→ git commit -m "Dodana datoteka README.md"

# Rezultat:

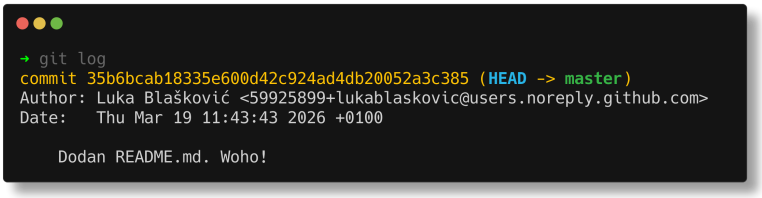
# [master (root-commit) 35b6bca] Dodan README.md. Woho!
# 1 file changed, 1 insertion(+)
# create mode 100644 README.md
```

- poruka je proizvoljna, ali dobra praksa je da bude informativna i sažeta, npr. “Dodana datoteka README.md” ili “Ažuriran sadržaj README.md”

Uspješno ste izradili commit koristeći Git CLI! Sada imate trajni zapis promjena u vašem projektu.

To možete provjeriti naredbom `git log` koja prikazuje povijest *commitova*:

```
→ git log
```

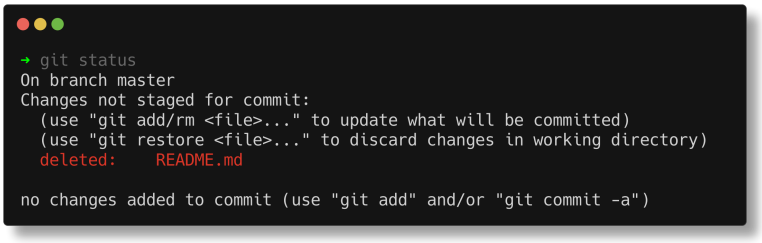


> Slika 19. Naredba `git log` prikazuje povijest *commitova*, uključujući poruke, datume i hash-eve *commitova*

Sada ćemo obrisati datoteku `README.md` i provjeriti status promjena nakon brisanja.

```
→ rm README.md

→ git status
```



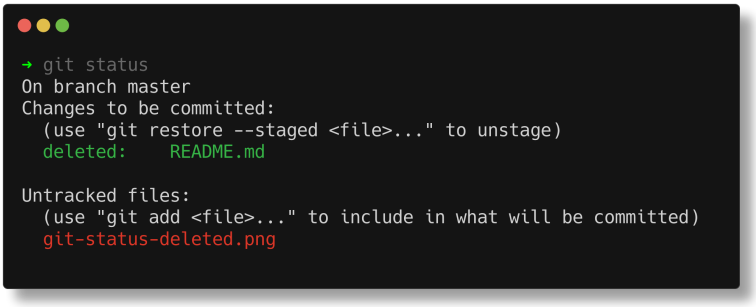
> Slika 20. Nakon brisanja datoteke `README.md`, naredba `git status` prikazuje da je datoteka obrisana, ali promjena još nije praćena (nije dodana u *staging area*)

Moramo dodati brisanje datoteke u *staging area* da bi se promjena zabilježila u sljedećem *commitu*:

```
→ git add .

# ili

→ git add README.md # bez obzira što je obrisana!
```



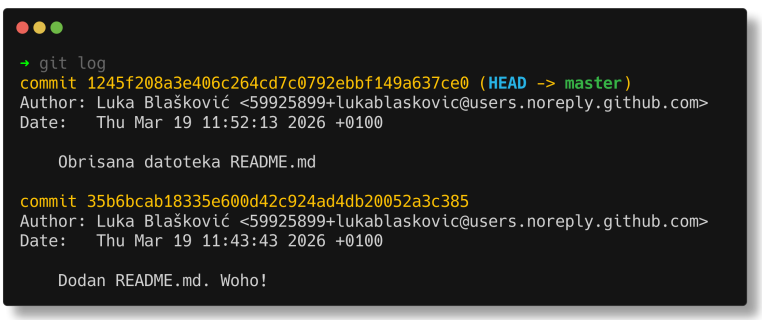
> Slika 21. Nakon dodavanja brisanja datoteke `README.md` u staging area, naredba `git status` prikazuje da je datoteka obrisana i promjena je sada praćena (*staged*)

Sada možemo stvoriti *commit* koji bilježi brisanje datoteke:

```
→ git commit -m "Obrisana datoteka README.md"
```

Provjerite povijest *commitova*. Trebali biste vidjeta oba.

```
→ git log
```



> Slika 22. Povijest *commitova* prikazuje oba *commita*: prvi za dodavanje datoteke `README.md`, a drugi za brisanje iste datoteke

Vraćanje na raniji commit

Glavna svrha Git-a je upravo **verzioniranje** (*eng. Software versioning*) i mogućnost vraćanja projekta na ranije verzije. Ovo je iznimno korisno kod rada na većim projektima, ali i kod manjih projekata kada želimo ispraviti grešku ili vratiti promjene koje su se pokazale problematičnima.

Sada smo samo zagrebali površinu Git CLI-a - ima tu još mnogo naredbi, opcija i mogućnosti koje ćemo obrađivati na sljedećim vježbama. Za sada je važno da razumijete osnovne koncepte i da ste se upoznali s Git CLI-jem kroz praktičan primjer.

Kako bismo vratili stanje našeg projekta na raniji *commit*, možemo koristiti naredbu `git checkout` ili `git reset`, ovisno o tome što želimo postići. Za sada ćemo pokazati kako koristiti `git checkout` za pregled ranijih *commitova*.

```
→ git checkout <hash_*commita*>
```

- `<hash_*commita*>` je jedinstveni identifikator *commita* koji se prikazuje u povijesti *commitova* (npr. `35b6bcab18335e600d42c924ad4db20052a3c385`)
- *hash vrijednost commita* možete kopirati iz naredbe `git log` ili drugog alata koji prikazuje povijest *commitova* (npr. GitHub Desktop)

Nakon izvršavanja `git checkout <hash_*commita*>`, vaš projekt će se vratiti na stanje koje je bilo u tom *commitu*. To znači da će datoteka `README.md` biti vraćena ako je bila prisutna u tom *commitu*, ili će biti obrisana ako je *commit* koji ste odabrali bio onaj nakon brisanja datoteke. Provjerite.

```
→ ls -a
```

Trebali biste vidjeti datoteku `README.md`.

Ako sada provjerite `git log`, vidjet ćete da ste i dalje na istoj grani (`master` ili `main`), ali se HEAD (pokazivač na trenutni *commit*) nalazi na ranijem *commitu*.

Super stvar je, što se sada **možete vratiti na prethodni *commit* naredbama**:

```
→ git checkout -
# ili
→ git switch - # nešto modernija sintaksa
```

Provjerite ponovno `git log` i `ls -a` da vidite da ste se vratili na najnoviji *commit* i da je datoteka `README.md` ponovno obrisana.

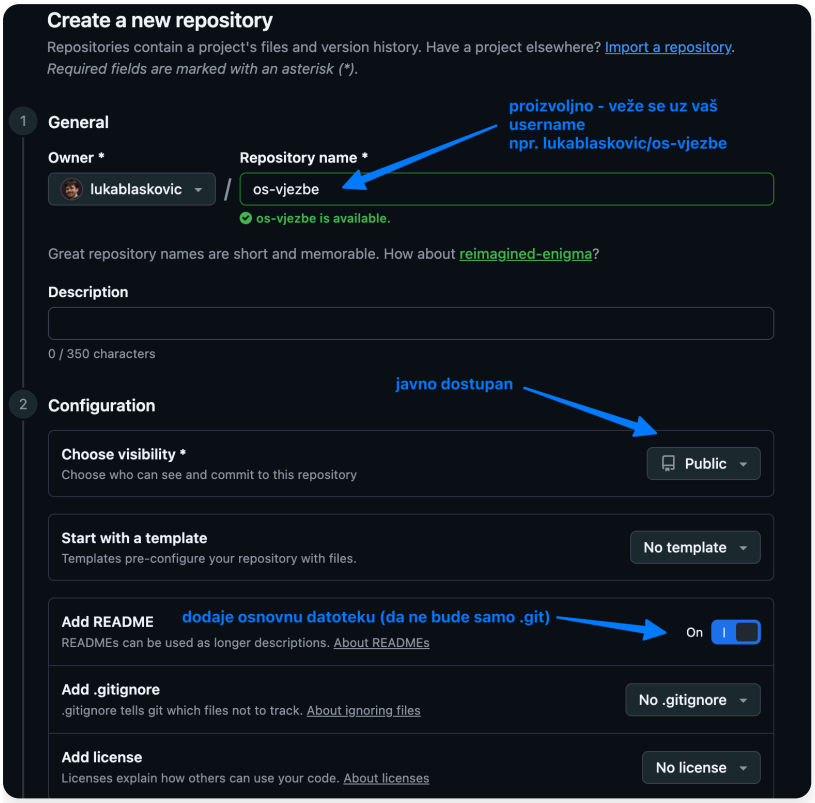
Toliko za sada! Pogledat ćemo još kako raditi s udaljenim repozitorijima (npr. GitHub, GitLab).

2. Način: Rad s udaljenim repozitorijem

Pokazat ćemo kako izraditi repozitorij na GitHub-u, klonirati ga lokalno i koristiti Git CLI za upravljanje promjenama.

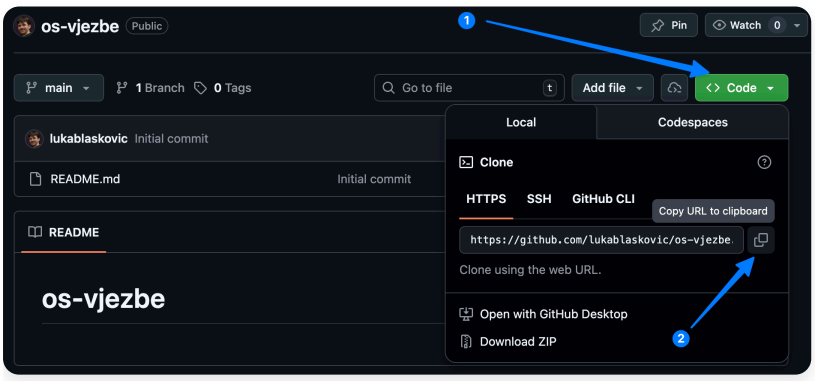
Otvorite GitHub račun ako ga već nemate.

Odaberite `Repositories` → `New` da biste stvorili novi repozitorij.



> Slika 23. Na GitHub-u možete stvoriti novi repozitorij klikom na `New`. Odaberite javni repozitorij i dodajte `README.md` datoteku.

Nakon što stvorite repozitorij, otvorite ga, odaberite `Code` i kopirajte URL repozitorija klikom na ikonu kopiranja.



> Slika 24. Kopirajte URL repozitorija s GitHub-a klikom na ikonu kopiranja.

To je to - dosta GUI-a! Sada se prebacujemo u terminal.

Dobra je praksa izraditi poseban direktorij za sve vaše Git projekte, npr. `~/git_projects` ili `~/GitHub` ili `~/faks-projects` - kako god želite. Stvorite taj direktorij ako ga već nemate:

```
→ mkdir ~/git_projects
→ cd ~/git_projects
```

Unutar njega, naredbom `git clone` klonirajte udaljeni repozitorij koji ste upravo stvorili na GitHub-u:

```
→ git clone <URL_repozitorija>

# Primjer:
→ git clone https://github.com/lukablaskovic/os-vjezbe.git
```

Na ovaj način, stvorit će se lokalna kopija repozitorija na vašem računalu, uključujući sve datoteke i povijest *commitova* pohranjen unutar `.git` direktorija. Glavna ideja je sinkronizacija između lokalne kopije i udaljenog repozitorija - promjene koje napravite lokalno možete "gurnuti" (*eng. push*) na GitHub, a promjene koje su napravljene na GitHub-u od strane drugih suradnika možete "povući" (*eng. pull*) u svoju lokalnu kopiju - na taj način možete učinkovito surađivati na projektima i imati dobru kontrolu nad verzijama.

Dakle, `git init` preskačemo jer smo klonirali gotovi repozitorij, ali sve ostale Git CLI naredbe koje smo ranije pokazali (npr. `git add`, `git commit`, `git log`, `git checkout`) su i dalje dostupne i rade na isti način unutar kloniranog repozitorija.

Provjerite sadržaj kloniranog repozitorija i *logove*:

```
→ cd os-vjezbe

→ ls -a

→ git log
```

Dodat ćemo direktorij `zadaca_02` i unutra stvoriti 4 datoteke: `zadatak_1.txt`, `zadatak_2.txt`, `zadatak_3.txt` i `zadatak_4.txt`.

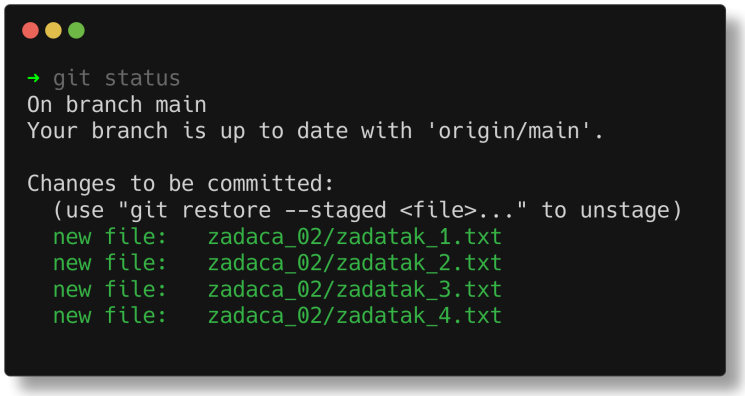
```
→ mkdir zadaca_02
→ cd zadaca_02
→ touch zadatak_1.txt zadatak_2.txt zadatak_3.txt zadatak_4.txt
```

Provjerite sadržaj direktorija i status promjena:

```
→ ls -a
→ git status
```

Kako ne biste svaku datoteku pojedinačno dodavali u *staging area*, jednostavno se prebacite u `zadaca_02` direktorij i dodajte sve datoteke odjednom:

```
→ cd zadaca_02
→ git add .
→ git status
```



> Slika 25. Nakon dodavanja svih datoteka u staging area, naredba `git status` prikazuje da su sve **datoteke praćene** i **spremne za *commit***.

Sada stvorite *commit* s porukom `"Dodani zadaci za Vježbu 2"`.

```
→ git commit -m "Dodani zadaci za Vježbu 2"

→ git log # provjera
```

Zadnji korak je tzv. *pushanje* odnosno **učitavanje lokalne kopije našeg repozitorija na GitHub**, kako bi promjene bile vidljive i dostupne online.

To radimo naredbom `git push`:

```
→ git push
```

Rezultat:

```
bash-3.2$ git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 14 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 391 bytes | 391.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/lukablaskovic/os-vjezbe.git
   0699efe..eea4fed  main -> main
```

Čestitam! Uspješno ste klonirali repozitorij, dodali nove datoteke, stvorili *commit* i pushali promjene na GitHub koristeći Git CLI!

Provjerite promjene na GitHub-u tako da otvorite repozitorij u pregledniku i vidite da su datoteke `zadatak_1.txt`, `zadatak_2.txt`, `zadatak_3.txt` i `zadatak_4.txt` sada prisutne, a *commit* "Dodani zadaci za Vježbu 2" je vidljiv u povijesti *commitova*. Sada možete nastaviti raditi na zadacima ispod i učitati ih na GitHub kada završite.

Sve ovo moguće je odraditi i kroz određene GUI alate (npr. GitHub Desktop, GitKraken, SourceTree), ali je važno da se upoznate i s Git CLI-jem jer vam daje potpunu kontrolu nad svim aspektima Git repozitorija - puno toga se ne može kroz GUI ili je ograničeno. Osim toga, mi ćemo aktivno koristiti i učiti Git CLI kroz sve vježbe, budući da je učenje CLI-ja jedan od temeljnih ishoda učenja na ovom kolegiju.

5.3 Zastavice Git CLI naredbi

Git CLI naredbe također imaju svoje zastavice koje omogućuju dodatne opcije i funkcionalnosti. Neke od najčešćih zastavica su:

Naredba	Primjer	Opis
`git init`	`git init -q`	Inicijalizira novi Git repozitorij. Zastavica `-q` (quiet) smanjuje količinu izlaznih informacija.
`git add`	`git add -A`	Dodaje sve promjene (uključujući brisanja) u staging area. Zastavica `-A` znači "all". Može se navesti i pojedinačna datoteka, npr. `git add README.md`.
`git commit`	`git commit -m "Poruka"`	Stvara *commit* s obaveznom porukom. Zastavica `-m` očekuje argument koji je poruka *commita*.
`git log`	`git log --oneline`	Prikazuje povijest *commitova*. Zastavica `--oneline` prikazuje sažeti prikaz *commitova*, gdje je svaki *commit* prikazan u jednom retku.

Naredba	Primjer	Opis
`git checkout`	`git checkout <hash_*commita*>`	Vraća stanje projekta na raniji *commit*. `<hash_*commita*>` je jedinstveni identifikator *commita*.
`git push`	`git push`	Pusha lokalne promjene na udaljeni repozitorij. Zastavice se obično ne koriste kod `git push`, ali se mogu koristiti za specificiranje grane ili udaljenog repozitorija, npr. `git push origin main`.
`git pull`	`git pull`	Povlači promjene s udaljenog repozitorija u lokalnu kopiju. Zastavice se obično ne koriste kod `git pull`, ali se mogu koristiti za specificiranje grane ili udaljenog repozitorija, npr. `git pull origin main`.
`git status`	`git status -s`	Prikazuje status promjena. Zastavica `-s` (short) prikazuje sažeti prikaz statusa, gdje su promjene prikazane u jednom retku s oznakama koje označavaju vrstu promjene.

Zadaci za Vježbu 2

Zadaću predajete na Merlin prema uputama za predaju.

Zadatke riješite izvršavanjem naredbi u kloniranom GitHub repozitoriju, tako da se vide rezultati. Sve naredbe i odgovore upišite u `vjezba_2.txt` datoteku.

Zadatak 1

Proučite **Git CLI** poglavlje i popratne naredbe iz 5. poglavlja. Izradite (ako već nemate) GitHub račun i stvorite novi repozitorij za vježbe (ako ga još nemate).

Koristeći Git CLI naredbe, klonirajte repozitorij na svoje računalo i pripremite **novi direktorij za druge vježbe**.

VAŽNO:

- Pokušajte riješiti zadatke u nastavku koristeći isključivo CLI, a nakon svakog riješenog zadatka stvorite commit s odgovarajućom porukom (npr. `"Riješen zadatak 1"`).
- *Pushajte* promjene na GitHub nakon rješavanja svakog zadatka.
- Jednom kada riješite sve zadatke, pokrenite naredbu `history` i kopirajte sve naredbe koje ste koristili od naredbe `git clone` pa nadalje, i zalijepite ih u `vjezba_2_history.txt` datoteku. Napravite novi commit s porukom `"Dodana povijest naredbi za Vježbu 2"` i *pushajte* promjene. Ovo možete napraviti i jednom naredbom koja preusmjerava sadržaj naredbe `history` u datoteku, npr. `history > vjezba_2_history.txt`.

Zadatak 2

U radnom direktoriju stvorite direktorije `vjezba_2/data` i `vjezba_2/backup` koristeći **dvije** naredbe `mkdir`.

Unutar `data` stvorite sljedeće datoteke:

```
script.js
style.css
.env
```

Naredbom `echo` unesite sljedeći sadržaj u datoteke:

- `script.js`: `console.log("Hello, World!")`
- `style.css`: `body { background-color: #f0f0f0; }`
- `.env`: `VJEZBE=OS`

Kojom naredbom možete provjeriti sadržaj datoteke u CLI-ju? Provjerite sadržaj svih datoteka.

Zadatak 3

Kopirajte sav sadržaj direktorija `data` u direktorij `backup`, uključujući i **skrivene datoteke**. Po potrebi upotrijebite više od jedne naredbe. Ispišite detalje o radnji.

Zatim se prebacite u direktorij `data` i napravite ispis sadržaja direktorija `../backup`. Ispis mora:

- biti detaljan,
- uključivati skrivene datoteke,

- biti sortiran po veličini (od najveće prema najmanjoj datoteci)

Zadatak 4

U direktoriju `data` izbrišite sve datoteke osim datoteke `.env`, pri čemu brisanje mora biti **interaktivno**. Ispišite detalje o radnji.

Nakon toga u direktorij `data` kopirajte sav sadržaj direktorija `backup` (samo sadržaj - bez direktorija), uključujući **skrivene datoteke**. Po potrebi upotrijebite jednu ili više naredbi, ali **spriječite prepisivanje datoteka koje već postoje**. Ispišite detalje o radnji.

Na kraju detaljno ispišite sadržaj direktorija `data` tako da ispis:

- uključuje skrivene datoteke,
- ne uključuje posebne direktorije `.` i `..`,
- bude sortiran prema datumu zadnje izmjene (od najnovije prema najstarijoj datoteci),
- prikazuje po jednu stavku u svakom retku.

Kojom još naredbom možete ispisati sadržaj direktorija `data`? Napišite naredbom i primjer rezultata.

Zadatak 5

Otvorite direktorij na vašem računalu po želji kroz CLI, ali neka ne sadrži više od 20 datoteka. Direktorij mora sadržavati ugniježđene direktorije s nekoliko datoteka unutar njih.

Prebacite se u direktorij i napišite sljedeće naredbe:

1. Ispišite detaljno sadržaj glavnog direktorija, uključujući sve skrivene datoteke, i sortirajte ga po veličini (od najveće prema najmanjoj datoteci).
2. Ispišite detaljno sadržaj glavnog direktorija bez skrivenih datoteka, sortirajte ga po veličini i prikažite jedinice uz veličinu datoteka.
3. Ispišite sav sadržaj direktorija, uključujući poddirektorije, njihove datoteke i skrivene datoteke koristeći stablastu strukturu.